

FRANKA PANDA

Bottle Pick, Shake, and Place

Engineering Reference Handbook

ROS 2 Jazzy | MoveIt 2 | PyMoveIt2 | Gazebo Harmonic

A function-by-function map of the final program, the V1 -> V2 -> V3 design evolution, all critical pose and trajectory calculations, the real troubleshooting history, and a reusable method for building manipulation tasks from scratch.

Prepared for Harivathsha

Project reference edition - 25 August 2026

How to use this handbook

This is not a line-by-line code commentary. Read it as an engineering design record. Start with the final-code flowcharts, use the calculation section while tuning geometry, and use the error ledger and decision tree when a future task fails.

Evidence labels

[CODE] is directly confirmed by one of the three verified source files. **[LOG]** is confirmed by recovered project chat or runtime output. **[INFERENCE]** is a technically reasonable explanation where the exact historical log was not recoverable. Missing evidence is stated rather than invented.

Version	Verified source	Role
V1	01_original_discrete_shake.py (1,494 lines)	Fixed collision bottle; four-corner X shake planned as separate motions
V2	Pasted code(9).py (1,966 lines)	Vision-synchronized bottle; continuous X-Z figure-eight; explicit cached start state
V3	03_long_fast_wrist_rock.py (2,210 lines)	Long vertical then horizontal strokes; wrist rock; fast retiming and settling

Important file correction

The earlier attachment named 02_continuous_figure_eight.py was a malformed, de-indented copy of V3 and is excluded from this analysis. Pasted code(9).py is the genuine V2 and compiles successfully.

Contents

How to use this handbook	2
Contents	3
1. System mental model: seven different kinds of state	5
2. Final program architecture and master flow	6
3. Function-by-function map of the final V3 code	9
3.1 Construction and orchestration	9
3.2 Perception and Planning Scene	9
3.3 Gazebo and MoveIt attachment	10
3.4 Generic motion, gripper, and placement	11
3.5 Quaternion and final shake engine	12
4. Geometry, pose, waypoint, and timing calculations	13
4.1 Final V3 parameter map	14
4.2 Frame assumption and target validation	15
4.3 Table and bottle collision geometry	15
4.4 Side-grasp pose derivation	15
4.5 Side-grasp quaternion calculation	16
4.6 V1, V2, and V3 positional path calculations	17
4.7 Wrist rocking and quaternion composition	18
4.8 Cartesian resolution and planning load	18
4.9 Time compression, velocity, acceleration, and endpoint hold	18
5. Engineering evolution: exactly what changed from V1 to V3	20
5.1 V1 -> V2: fix world-model consistency and motion continuity	21
5.2 V2 -> V3: increase mixing action without breaking planning/execution	22
5.3 What was deliberately preserved through all versions	22
6. Error archaeology: symptom -> subsystem -> correction	23
6.1 Geometry, perception, collision, and attachment errors	23
6.2 Trajectory planning, timing, and controller errors	24
6.3 Build/environment error: correct source but old installed program	25
7. Reusable engineering method for building from scratch	26
7.1 Design around invariants, not around motion commands	26
7.2 The first-failure rule	27
7.3 Safe one-variable experimental method	27
7.4 Recommended test record	27

7.5 Geometry-first decision rule 27

7.6 Mechanical interpretation of the fast shake 28

8. What remains fragile in the final V3 code 29

8.1 Which issues are acceptable for this simulation demonstration? 29

9. Blueprint for the next manipulation program 31

9.1 Design checklist 31

9.2 Ten questions that test real understanding 31

10. Glossary and source notes 32

10.1 Evidence and limitations 32

1. System mental model: seven different kinds of state

Most manipulation bugs come from confusing a requested pose with the state that another subsystem currently believes. The final program deliberately keeps these layers separate.

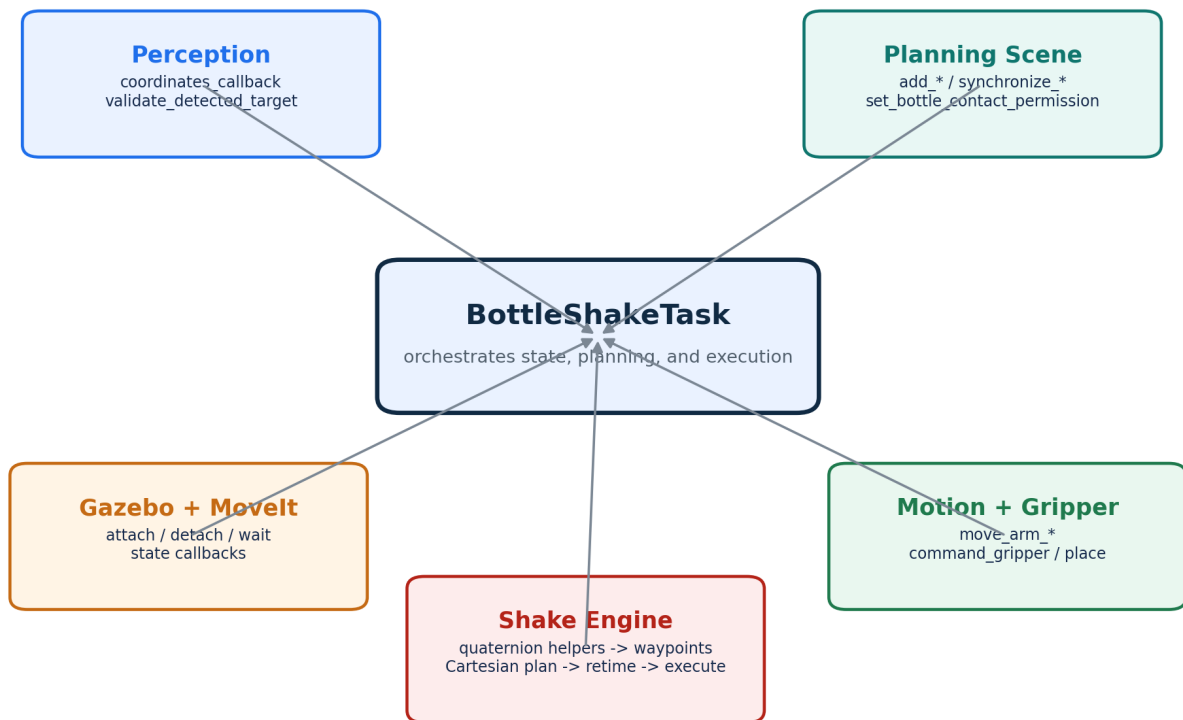
Layer	Meaning	Owner	How it can disagree
Desired Cartesian pose	Requested hand X/Y/Z plus quaternion	BottleShakeTask	A correct-looking pose may be unreachable or in the wrong frame
Sampled waypoints	Many desired Pose messages forming the shake	BottleShakeTask	A curve can be mathematically smooth but kinematically difficult
IK joint states	Seven Panda angles solving each pose	Movelt	IK branches can jump or hit joint/collision limits
Timed joint trajectory	Joint positions plus time, velocity, acceleration	Movelt/TOTG then V3 retiming	Timestamps can be invalid or limits can be exceeded after compression
Controller state	Actual/commanded joint motion and /joint_states	ros2_control	Cached state may lag the physically simulated joints
Gazebo bottle	Physical simulated pose and attachment constraint	Gazebo	Bottle may move physically while Movelt still sees a world object
Movelt bottle	Collision cylinder in world or attached robot state	Planning Scene	Planning state can disagree with camera/Gazebo

Core invariant

Before every irreversible-looking stage, confirm that the subsystem which owns the required state agrees: perception before planning, Planning Scene before approach, Gazebo plus Movelt before lift, controller state before a new trajectory, and both detachment systems before release.

2. Final program architecture and master flow

Final-Code Function Architecture



The class is a procedural state machine. It is not formally represented by an enum, but every helper returns True/False and the caller immediately aborts on False. This makes the first failed stage the most useful diagnostic evidence.

Final Task Flow - Initialization and Pick

Every stage returns False immediately when its required invariant is not satisfied.



Final Task Flow - Transport, Shake, and Place

The shake is one trajectory; pick/place remain guarded stage-by-stage.



3. Function-by-function map of the final V3 code

The maps below cover all 30 **def** statements in V3—including the nested **append_stage** helper—and answer five questions: what enters, what changes, who calls it, what failure means, and why the function deserves to exist.

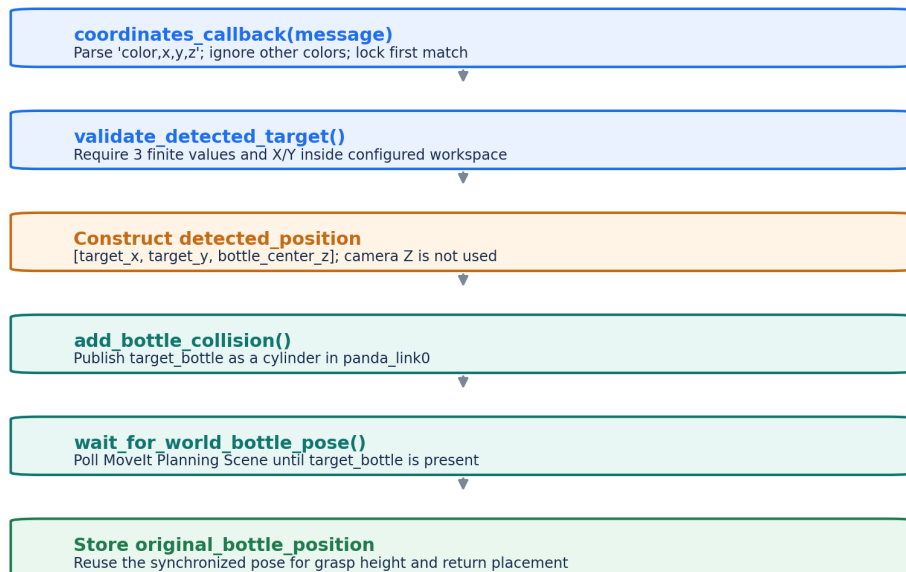
3.1 Construction and orchestration

Function	Called by	Purpose and output	Why an engineer keeps it separate
<code>__init__</code>	main	Declare parameters; create MoveIt, action, service, publisher and subscription interfaces; initialize IDs, states and poses.	One place establishes configuration and ROS graph dependencies before task logic runs.
main	Python entry point	Initialize ROS; build static scene; wait for vision; synchronize bottle; move START; execute task; clean shutdown.	Prevents the task from running before its external world model is ready.
execute_day1_pick_place	main	Calculate grasp/pre-grasp/lift poses and call the guarded HOME -> PICK -> LIFT -> SHAKE -> PLACE sequence.	Central state machine exposes the first failing stage and preserves task order.

3.2 Perception and Planning Scene

Perception and Planning-Scene Synchronization

Key invariant: the grasp target and MoveIt collision object describe the same bottle.



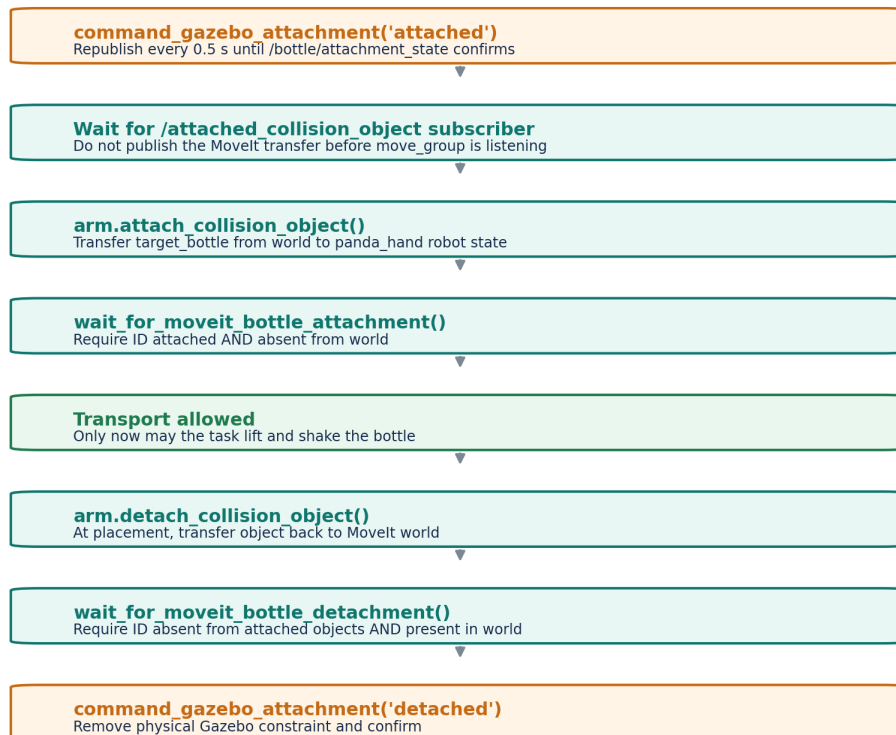
Function	Input -> output	Failure meaning	Engineering reason
coordinates_callback	String color,x,y,z -> locked target_coords	Bad field count/type; requested color not yet present	Freezes one target so later stages do not chase camera jitter.
validate_detected_target	target_coords -> accepted/rejected	Missing length, NaN/Inf, X/Y outside [0.40,0.75] x [-0.30,0.30]	Reject obviously unsafe data before asking IK/planning to diagnose it indirectly.
wait_for_planning_scene_subcriber	Timeout -> bool	move_group is not listening to /collision_object	Avoid losing asynchronous object publications during startup.
add_table_collision	Fixed size/pose -> work_table box	Publication itself has no acknowledgement	Gives MoveIt the obstacle that defines the task surface.
add_bottle_collision	3-D position -> target_bottle cylinder	Wrong vector length raises ValueError	Simplifies the payload into conservative collision geometry.

Function	Input -> output	Failure meaning	Engineering reason
wait_for_world_bottle_pose	Expected position -> scene-presence bool	Object never appears in MoveIt world	Confirms MoveIt consumed the asynchronous update before motion begins.
synchronize_bottle_with_vision	Locked target + bottle_center_z -> original_bottle_position	Validation or scene confirmation fails	Makes camera grasp X/Y and MoveIt cylinder X/Y refer to the same object.
remove_bottle_collision	Object ID -> removal request	No explicit confirmation	Maintenance utility; not used by the final successful sequence.
add_static_environment_collision_objects	None -> table ready	Subscriber unavailable	Separates fixed geometry (table) from dynamic geometry (vision bottle).
set_bottle_contact_permission	Other names + allowed flag -> updated ACM	Service unavailable, malformed matrix, timeout or rejection	Expected grasp/table contacts must be allowed without disabling global collision checking.

3.3 Gazebo and MoveIt attachment

Dual Attachment State Machine

Gazebo owns physical motion; MoveIt owns collision/planning state.



Function	Purpose	Success condition	Why needed
attachment_state_callback	Store attached/detached feedback from Gazebo	Only known state strings accepted	Command publication alone is not evidence of physical simulation state.
command_gazebo_attachment	Retry attach/detach command until feedback or timeout	Bridge subscriber exists and requested state is observed	Handles startup races and lost one-shot messages.
wait_for_moveit_bottle_attachment	Poll attached and world object lists	target_bottle attached AND absent from world	Prevents duplicate or stale planning geometry before lift.
wait_for_moveit_bottle_detachment	Poll reverse state	target_bottle absent from attached AND present in world	Confirms Planning Scene is ready for release and retreat.

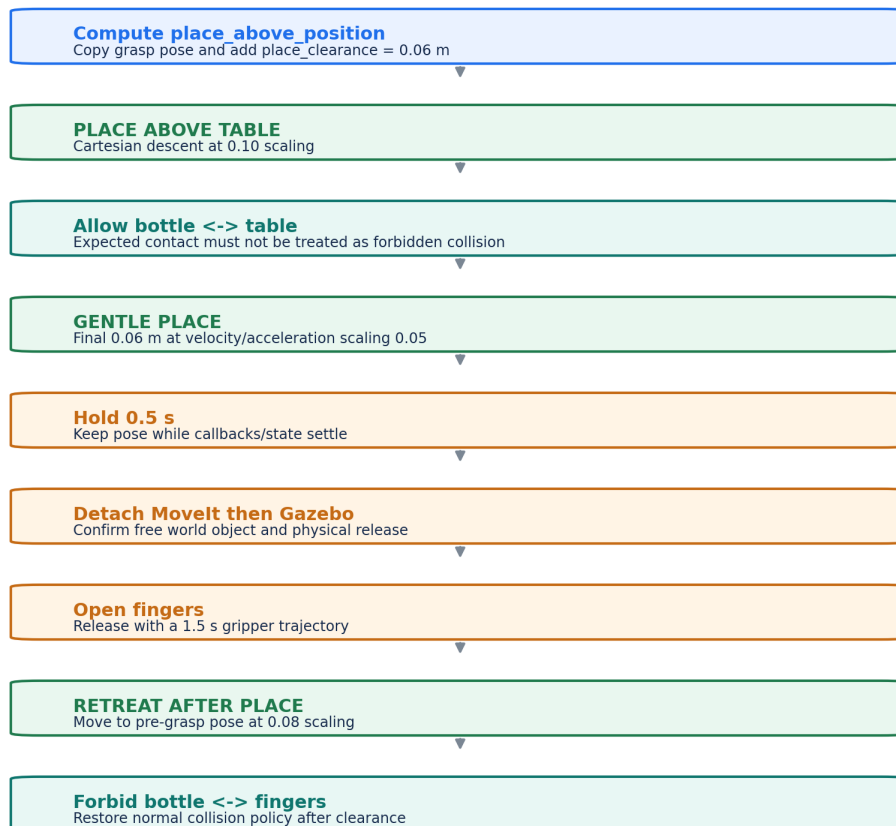
Function	Purpose	Success condition	Why needed
attach_bottle_for_transport	Gazebo attach -> MoveIt attach -> verify	Both physical and planning representations agree	A payload must move physically and be included in collision planning.
detach_bottle_after_transport	MoveIt detach -> Gazebo detach -> verify	Both representations agree on free bottle	Avoids opening fingers while either subsystem still models transport.

3.4 Generic motion, gripper, and placement

Function	What it does	Checks	Why separate
move_arm_to_joints	Plan named seven-joint target and execute	Nonempty trajectory; wait_until_executed success	Known configurations such as START/HOME do not need Cartesian pose solving.
move_arm_to_pose	Plan free-space or Cartesian hand pose	Nonempty plan; execution result; configured tolerances/fraction	One wrapper keeps planning/execution logs and fail-fast behavior consistent.
command_gripper	Send one two-finger FollowJointTrajectory goal	Server, timeout, acceptance, result, ROS status and controller error code	The arm and gripper use different controllers; publication is not proof of closure.
place_bottle_gently	Two-speed descent, contact permission, detach, release, retreat	Every motion, ACM, detach and gripper stage	Placement is a state transition, not a single pose command.

Gentle Placement Flow

The table contact permission remains appropriate because the free bottle rests on the table.



3.5 Quaternion and final shake engine

Final Long-Fast Shake Pipeline

Desired poses -> IK joint states -> timed trajectory -> controller execution.



Function	Input -> output	Guard/failure	Why introduced
normalize_quaternion	4 components -> unit quaternion	Norm below 1e-9 raises ValueError	Orientation math requires normalized rotations; prevents invalid zero orientation.
multiply_quaternions	left, right -> left * right	Assumes four components	Composes base side-grasp orientation with a local wrist rotation correctly.
wrist_rock_orientation	Base orientation + local-Z angle -> quaternion	Normalizes composed result	Changing quaternion components directly would not represent a clean axis rotation.
build_long_fast_shake_waypoints	Centre, amplitudes, cycles, samples -> 192 Pose waypoints	Upstream validation	Its nested append_stage(axis, amplitude) reuses one 96-pose phase sampler for the vertical and horizontal stages.
execute_continuous_mixing_path	Pose list -> validated, compressed joint trajectory -> execution bool	Service, error code, fraction, point count, time, controller	Concentrates all expensive and safety-critical trajectory checks in one transaction.
perform_mixing_shake	Task parameters -> complete shake bool	Valid ranges for sizes, cycles, scaling, time scale, step, hold and wrist angle	Boundary function rejects unreasonable configuration before generating thousands of IK samples.

4. Geometry, pose, waypoint, and timing calculations

This chapter converts every important final value into a physical meaning. Use the equations, not copied coordinates, when adapting the task to a new robot, table, bottle, frame, or gripper.

Calculation family	What this chapter derives	Why it matters
Collision geometry	Table top, bottle bottom/top, diameter and nominal overlap	Explains ACM table contact and model alignment
Side-grasp geometry	Grasp, pre-grasp, lift and place-above poses from x_b/y_b	Turns trial-and-error coordinates into reusable equations
Orientation	Side-grasp rotation matrix and local-Z quaternion composition	Connects tool axes to approach and wrist rock
Waypoints	17 V1 transactions, 128 V2 poses and 192 V3 poses	Separates requested samples from generated trajectory points
Timing	0.55 time scale, 1.818x velocity and 3.306x acceleration	Shows why the final speedup is simulation-only

4.1 Final V3 parameter map

Parameter	Final value	Unit/type	Physical meaning	If increased
approach_offset	0.12	m	Extra stand-off from grasp hand pose	Safer clearance but longer approach and possibly harder reach
cartesian_fraction_threshold	0.98	ratio	Minimum accepted fraction for generic Cartesian moves	Stricter completion requirement
table_size	[0.80,1.20,0.10]	m	Collision-box dimensions	More conservative obstacle; may reject valid paths
table_position	[0.60,0.00,-0.10]	m	Box centre in panda_link0	Depends on axis; moves entire collision surface
bottle_center_z	0.04	m	Trusted upright bottle-cylinder centre Z	Raises collision object, grasp and return height
workspace_x_limits	[0.40,0.75]	m	Accepted detected bottle-centre X	Wider acceptance, not necessarily safe shake reach
workspace_y_limits	[-0.30,0.30]	m	Accepted detected bottle-centre Y	Wider acceptance, not derived-pose validation
bottle_height	0.20	m	Collision-cylinder height	More conservative vertical payload
bottle_radius	0.035	m	Cylinder radius; diameter 0.070 m	More clearance required around obstacles/robot
side_grasp_depth	0.105	m	Bottle-centre to panda_hand offset along -Y	Hand target moves farther from bottle
lift_distance	0.35	m	Vertical rise from grasp to shake centre	More table clearance; higher reach/load
shake_horizontal_amplitude	0.12	m	Side-to-side X excursion	Wider stroke; more reach/inertia
shake_vertical_amplitude	0.15	m	Up/down Z excursion	Longer stroke; more gravity/reversal load
shake_cycles	3	cycles/stage	Three vertical plus three horizontal cycles	Longer total shake and more waypoints
shake_samples_per_cycle	32	poses/cycle	Requested pose sampling density	Smoother request but more IK/planning load
shake_wrist_rotation_degrees	45	deg	Local-Z wrist angle amplitude	Stronger rotation; greater joint-7 demand
shake_velocity	1.0	scale	100% of configured MoveIt velocity limits	Cannot normally exceed 1.0 through scaling
shake_acceleration	1.0	scale	100% of configured MoveIt acceleration limits	Cannot normally exceed 1.0 through scaling
shake_time_scale	0.55	ratio	Execute path in 55% of planned time before hold	Larger is slower/safer; smaller is faster/riskier
shake_cartesian_max_step	0.01	m	Max Cartesian interpolation step for shake	Fewer IK samples and faster planning, coarser checking
shake_endpoint_hold_seconds	0.45	s	Stationary final trajectory point	More settling time; longer task
place_clearance	0.06	m	Height above grasp pose where gentle stage begins	Longer slow descent
place_velocity/acceleration	0.05 / 0.05	scale	Final contact-motion scaling	Faster contact and higher placement disturbance
gripper_open	[0.040,0.040]	m joints	Approximately 80 mm total symmetric opening	Up to configured gripper limit
gripper_grasp	[0.035,0.035]	m joints	Approximately 70 mm commanded gap for 70 mm bottle	Larger gap gives less/zero geometric closure

Scaling is not a physical speed

A value such as 0.30 or 1.0 multiplies the velocity/acceleration limits configured for every joint. The limiting joint, IK solution, path curvature, time parameterizer, and controller determine the actual end-effector speed. It is not 0.30 m/s or 1 rad/s.

4.2 Frame assumption and target validation

[CODE] Every collision object and pose is expressed in **panda_link0**. The String on /color_coordinates contains no frame ID and the script performs no TF transform. Therefore the perception node must already publish X/Y/Z in the Panda base frame. Copying camera-optical-frame coordinates directly would move along the wrong axes.

```
target accepted when:
  0.40 <= x_b <= 0.75
  -0.30 <= y_b <= 0.30
  x_b, y_b, z_camera are finite

MoveIt bottle pose used by the task:
  bottle = [x_b, y_b, 0.04]
```

Subtle limitation

The workspace check applies only to the detected bottle centre. It does not test the derived pre-grasp pose, the -0.105 m hand offset, or the +/-0.12 m shake extremes. A bottle accepted at x=0.40 could request a horizontal shake as low as x=0.28. Final reachability is still delegated to MoveIt.

4.3 Table and bottle collision geometry

Quantity	Calculation	Result	Meaning
Table top Z	$-0.10 + 0.10/2$	-0.05 m	Top face of work_table box
Bottle bottom Z	$0.04 - 0.20/2$	-0.06 m	Bottom face of target_bottle cylinder
Bottle top Z	$0.04 + 0.20/2$	+0.14 m	Top of collision cylinder
Nominal overlap	$-0.05 - (-0.06)$	0.01 m	Cylinder extends 1 cm into conservative table geometry
Bottle diameter	2×0.035	0.070 m	Matches commanded 70 mm gripper gap geometrically

[INFERENCE] The 1 cm overlap makes visual contact robust against small model offsets, but it also means MoveIt sees bottle-table collision. This explains why the ACM temporarily/appropriately allows target_bottle <-> work_table during attachment and placement. It is a planning permission, not a Gazebo friction change.

4.4 Side-grasp pose derivation

Let the validated bottle centre be $B = [x_b, y_b, 0.04]$. The code does not aim the panda_hand frame at the bottle centre. It offsets the hand so the finger contact region surrounds the cylinder.

```
grasp_height = bottle_center_z + 0.05
              = 0.04 + 0.05 = 0.09 m

grasp_position   G = [x_b, y_b - 0.105, 0.09]
pre_grasp_position P = [x_b, y_b - 0.105 - 0.12, 0.09]
                  [x_b, y_b - 0.225, 0.09]

lift_position     L = [x_b, y_b - 0.105, 0.09 + 0.35]
                  [x_b, y_b - 0.105, 0.44]

place_above      [x_b, y_b - 0.105, 0.09 + 0.06]
                  [x_b, y_b - 0.105, 0.15]
```

Pose	X	Y	Z	Engineering intention
Bottle centre	x_b	y_b	0.04	Synchronized collision object
Pre-grasp hand	x_b	y_b - 0.225	0.09	12 cm clearance before approach
Grasp hand	x_b	y_b - 0.105	0.09	Calibrated hand-frame offset around bottle
Lift/shake centre	x_b	y_b - 0.105	0.44	35 cm vertical table clearance
Place above	x_b	y_b - 0.105	0.15	6 cm above final contact pose

Why repeated lowering happened

[LOG] The project history confirms repeated grasp-height/coordinate corrections and symptoms such as stopping too high/low or ahead/behind. [INFERENCE] Those symptoms are exactly what frame disagreement, hand-frame offsets, collision-object mismatch, or an incorrect trusted Z produce. The exact old coordinate sequence is not fully recoverable, so this handbook does not invent a test-by-test chronology.

4.5 Side-grasp quaternion calculation

The orientation $q = [-0.5, -0.5, -0.5, 0.5]$ is in XYZW order. Its norm is $\sqrt{4 \times 0.25} = 1$, so it is already a unit quaternion. Its rotation matrix is:

```
R = [[0, 1, 0],
      [0, 0, 1],
      [1, 0, 0]]
```

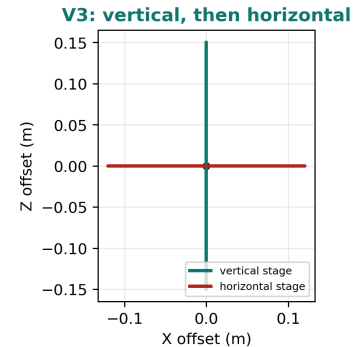
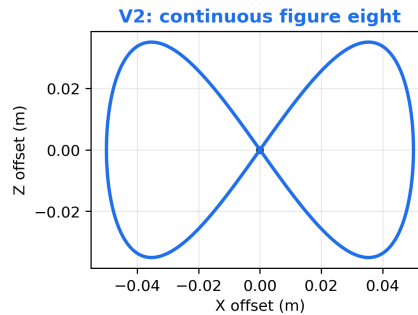
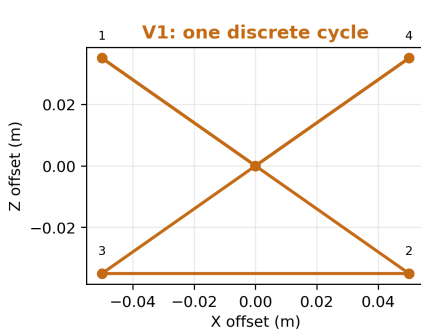
Columns describe local hand axes in world/base coordinates:

```
hand +X -> world +Z (jaws remain vertical)
hand +Y -> world +X (finger-opening direction)
hand +Z -> world +Y (approach direction)
```

Because pre-grasp subtracts 0.12 from world Y and approach moves toward larger Y, the hand's local +Z approach axis is aligned with the actual Cartesian approach direction.

4.6 V1, V2, and V3 positional path calculations

Position Paths Relative to the Lift Centre



V1: four corner targets, planned separately

```
Upper-left = centre + [-0.05, 0, +0.035]
Lower-right = centre + [+0.05, 0, -0.035]
Lower-left = centre + [-0.05, 0, -0.035]
Upper-right = centre + [+0.05, 0, +0.035]
```

```
4 corners x 4 cycles = 16 plan/execute/wait transactions
+ 1 return-to-centre transaction = 17 independent motions
```

Every independent motion is time-parameterized as its own trajectory and the code waits for completion. The arm therefore loses continuity and visibly pauses even if the scaling factors are raised.

V2: one continuous X-Z figure eight

```
samples = cycles x samples_per_cycle = 4 x 32 = 128 poses
phi_n = 2*pi*n/32, n = 1..128

x_n = x_c + 0.05*sin(phi_n)
y_n = y_c
z_n = z_c + 0.035*sin(2*phi_n)
orientation = constant side-grasp quaternion
```

At $n=128$, $\phi=8\pi$; both sine terms are zero, so the final requested pose is the exact centre. [LOG] One recovered successful plan reported fraction=1.000, 147 generated trajectory points, duration=14.59 s, and average_frequency=4/14.59=0.274 Hz (logged as 0.27 Hz). Requested waypoints and generated joint-trajectory points are not the same count.

V3: long vertical stage followed by long horizontal stage

```
samples per stage = 3 cycles x 32 = 96 poses
two stages        = 96 vertical + 96 horizontal = 192 poses

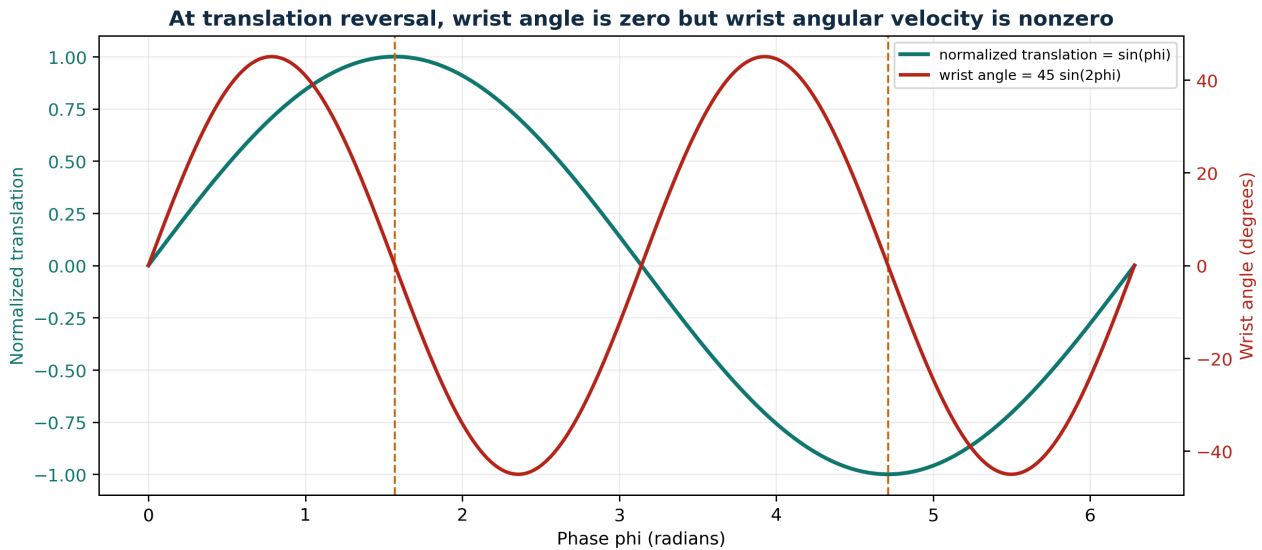
Vertical stage:  x = x_c, y = y_c, z = z_c + 0.15*sin(phi)
Horizontal stage: x = x_c + 0.12*sin(phi), y = y_c, z = z_c

At the default lift centre z_c = 0.44:
    vertical minimum = 0.44 - 0.15 = 0.29 m
    vertical maximum = 0.44 + 0.15 = 0.59 m

Horizontal range:
    x_min = x_b - 0.12
    x_max = x_b + 0.12
```

At sample 96, $\phi=2\pi*96/32=6\pi$; $\sin(\phi)=\sin(2\phi)=0$. Therefore the vertical stage ends at its centre/base orientation. The horizontal stage also ends exactly at centre, which prepares placement without a separate return motion.

4.7 Wrist rocking and quaternion composition



```
wrist amplitude theta_max = 45 deg
theta(phi) = 45 deg * sin(2*phi)

local-Z rotation quaternion:
  q_r = [0, 0, sin(theta/2), cos(theta/2)]

composed orientation:
  q_waypoint = normalize(q_base * q_r)
```

At a positional reversal, $\phi = \pi/2$ or $3\pi/2$. Translation $\sin(\phi)$ is at ± 1 and its velocity is zero. Wrist angle $\sin(2\phi)$ is zero, but $d[\sin(2\phi)]/d\phi = 2\cos(2\phi)$ is nonzero. The wrist passes smoothly through zero instead of demanding a simultaneous hard orientation reversal. This was introduced after the documented Movelt/TOTG 'path requires a 180 degree turn' problem.

For a $+45$ deg local-Z example, $\sin(22.5 \text{ deg}) = 0.382683$ and $\cos(22.5 \text{ deg}) = 0.923880$. Composing with the base quaternion gives approximately $[-0.653282, -0.270598, -0.270598, 0.653282]$, which remains unit length after normalization.

4.8 Cartesian resolution and planning load

Version	Shake max_step	Meaning	Consequence
V1/V2	0.0025 m	About 2.5 mm between Cartesian interpolation checks	Dense IK work; V2 long path produced a noticeable delay
V3	0.01 m	About 10 mm maximum step in open-air shake	4x larger step; roughly one-quarter interpolation samples for equal path length

The one-quarter statement is an approximation: Movelt may add points because of waypoint boundaries, IK changes and time parameterization. `max_step` controls geometric interpolation resolution, not controller cycle distance.

4.9 Time compression, velocity, acceleration, and endpoint hold

```
shake_time_scale s = 0.55

new time:      t' = s*t      = 0.55*t
speedup:       1/s           = 1.818x
new velocity:  v' = v/s       = 1.818*v
new acceleration: a' = a/s^2   = a/0.3025 = 3.306*a

hold timestamp = final compressed motion time + 0.45 s
hold positions = final positions
hold velocity  = 0
hold acceleration = 0
```

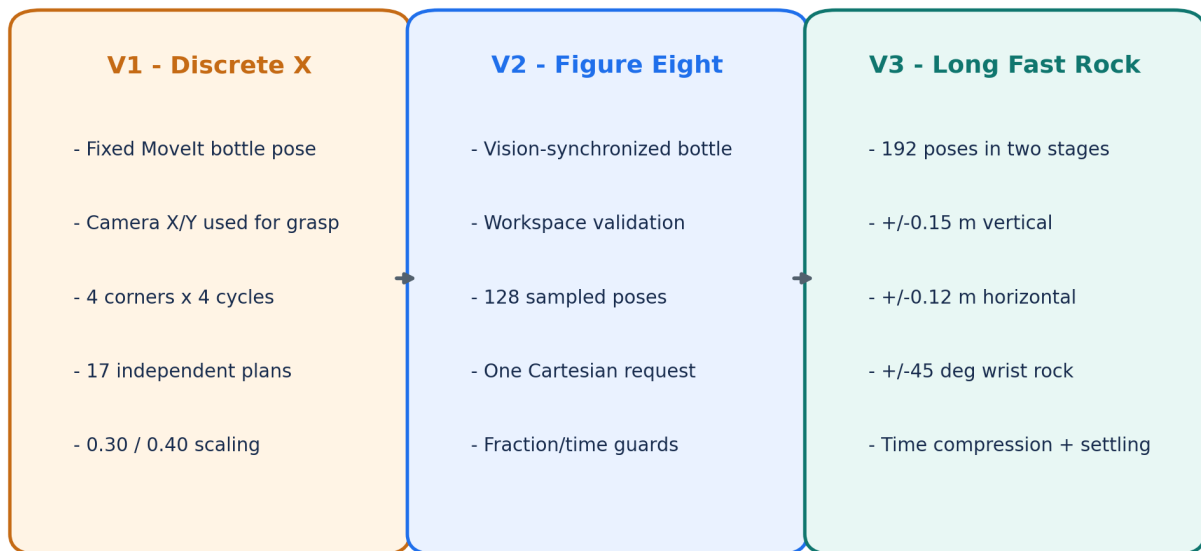
Simulation-only risk

Movelt scaling already stops at 1.0. Manual time compression can demand velocities and accelerations above configured Panda limits even though the geometric path is unchanged. The final code validates timestamps but does not perform a new per-joint limit audit after compression. Treat 0.55 as a Gazebo experiment, not a hardware-safe setting.

The 0.45 s endpoint hold gives panda_joint7 time to converge at the base grasp orientation. After controller success, a further 0.30 s ROS spin processes final /joint_states before placement. Before the shake request, 0.25 s is used to process the post-lift state and request.start_state is intentionally left empty so move_group snapshots its current state.

5. Engineering evolution: exactly what changed from V1 to V3

Evolution: V1 -> V2 -> V3



Each version preserves working safety logic and changes the weakest subsystem first.

Dimension	V1 - discrete X	V2 - figure eight	V3 - long fast wrist rock
Bottle scene pose	Fixed [0.60,0.00,0.04]	Detected X/Y + trusted Z	Same synchronized design
Target validation	Format/color only	Finite X/Y/Z + workspace X/Y	Same
Original placement pose	Fixed bottle_position assumptions	Stored original_bottle_position	Same
Shake request	17 plan/execute calls	128 poses, one Cartesian call	192 poses, one Cartesian call
Position path	Four X-pattern corners	X-Z figure eight	Vertical first, horizontal second
Orientation	Fixed quaternion	Fixed quaternion	+/-45 deg local-Z rock
Shake scaling	0.30 vel / 0.40 accel	0.30 / 0.40	1.0 / 1.0
Interpolation	0.0025 m per small segment	0.0025 m across long path	0.01 m for open-air shake
Start state	PyMoveIt2 helper per segment	Explicit cached self.arm.joint_state	Empty request after 0.25 s sync
Retiming	MoveIt only	MoveIt only	0.55 time scale + velocity/acceleration rescale
End settling	Every segment naturally stops	No explicit final hold	0.45 s hold + 0.30 s state sync

5.1 V1 -> V2: fix world-model consistency and motion continuity

Previous weakness / symptom	Root cause	V2 modification	Why it works	Trade-off / diagnostic lesson
Fixed target_bottle could disagree with camera grasp X/Y [CODE]	Planning Scene and task pose described different bottles	Replace bottle_position with bottle_center_z; create cylinder after detection at target X/Y	Collision checking, approach and placement share the same target	Still assumes camera data is already in panda_link0; inspect frame before tuning offsets
Invalid/unreachable-looking detections could enter planning [CODE]	String parsing only checked type/field count	validate_detected_target checks finite values and X/Y workspace	Rejects bad data at the correct subsystem boundary	Limits only validate centre; derived poses and shake extremes still need planning
Placement depended on fixed geometry [CODE]	Original bottle pose was not recorded from the dynamic target	Store original_bottle_position after scene confirmation	Grasp Z and return placement reuse one synchronized reference	If bottle slips after grasp, stored pose is no longer the physical truth
Four-corner shake stopped between targets [CODE/LOG]	Each corner was planned, executed and awaited independently	build_figure_eight_waypoints creates 128 poses; execute_continuous_mixing_path sends one request	Controller receives a continuous timed joint trajectory	Long request costs more IK/time-parameterization work
No whole-shake completion contract [CODE]	Only small segments were individually checked	Require GetCartesianPath SUCCESS and fraction ≥ 0.999	Never execute a path that stops away from expected centre	Strict threshold may reject a mostly valid path; solve geometry rather than lowering it blindly
Controller-time problems difficult to localize [CODE]	No explicit inspection of complete trajectory timing	Reject empty points, non-increasing timestamps and nonpositive duration	Separates planning success from executable trajectory validity	Validation detects but does not repair a broken time parameterization
Post-lift start needed a known state [CODE]	Cartesian service requires a start robot state	Wait for self.arm.joint_state and assign request.start_state.joint_state	At first, this made the request explicit and diagnosable	The cached value later proved capable of lagging the controller

Engineer pattern learned

First repair the state representation, then improve the trajectory. A smooth path cannot compensate for a bottle collision object at the wrong coordinates, and perfect perception cannot compensate for 17 stop-start controller transactions.

5.2 V2 -> V3: increase mixing action without breaking planning/execution

Observed limitation	Cause	V3 modification	Why it works	New risk / lesson
Figure eight was smooth but not the requested vigorous action [LOG]	Short +/-0.05 m X and +/-0.035 m Z path with fixed wrist	Replace with +/-0.15 m vertical stage, then +/-0.12 m horizontal stage	Long strokes create larger accelerations and visibly stronger mixing	Reach, inertia, bottle moment and collision clearance become more demanding
Friend suggested using rotating gripper [LOG]	Translation alone underused Panda's redundant wrist motion	Add +/-45 deg local-Z wrist rock throughout both stages	Combines translation and rotation while keeping side grasp	panda_joint7 tracking becomes critical; gripping moment increases
Abrupt orientation design triggered 'path requires a 180 degree turn' [LOG]	Hard orientation cusp is difficult for time parameterization	Quaternion composition with $\theta = 45 \sin(2\phi)$	Orientation varies continuously; at translation reversal wrist passes through zero with nonzero angular velocity	More complex IK path; quaternion must remain normalized
Long pause after lift; thousands of unnecessary IK samples [LOG/CODE]	0.0025 m resolution applied to a long dense Cartesian request	shake_cartesian_max_step = 0.01 m	4x coarser step substantially reduces interpolation workload	Coarser geometry/collision sampling; appropriate only because shake is in open space
Shake remained slow even at MoveIt scaling 1.0 [LOG]	Scaling cannot exceed configured limit and time parameterization remained conservative	Set 1.0/1.0 then compress timestamps by 0.55	Same geometric path executes about 1.82x faster before hold	Can exceed real/configured velocity and acceleration limits; simulation-only
Start point deviated >0.01 at panda_joint2/joint7 [LOG]	Copied self.arm.joint_state could be stale relative to move_group/controller	Spin 0.25 s and leave request.start_state empty	move_group snapshots its own current state when handling the request	Fixed wait is heuristic, not proof of state convergence
Placement failed immediately after fast wrist shake [LOG]	Final wrist had not fully settled / latest joint state had not propagated	Append 0.45 s stationary endpoint; spin 0.30 s after execution	Gives joint7 time to converge and lets fresh /joint_states reach the next plan	No explicit numeric convergence check; only time-based settling
Retimed timestamps might become invalid [CODE]	Nanosecond rounding or modification could break monotonicity	Validate timestamps before and after compression/hold insertion	Controller never receives equal/decreasing time_from_start	Still needs per-joint limit validation for hardware

5.3 What was deliberately preserved through all versions

- Fail-fast stage sequencing: later stages never pretend an earlier state transition succeeded.
- Free-space PRE-GRASP and Cartesian APPROACH/LIFT/PLACE/RETREAT.
- Direct FollowJointTrajectory gripper control with acceptance/result checks.
- Planning Scene table and bottle collision models.
- ACM permissions for expected bottle-finger and bottle-table contacts.
- Separate Gazebo physical attachment and MoveIt planning attachment, both verified.
- Two-speed gentle placement and restoration of previous velocity/acceleration settings.
- Same side-grasp orientation and calibrated side_grasp_depth/lift geometry.

6. Error archaeology: symptom -> subsystem -> correction

Use this ledger by locating the first stage that failed. Do not start by changing random coordinates or tolerances; identify which subsystem owns the failed invariant.

6.1 Geometry, perception, collision, and attachment errors

Symptom / exact evidence	Stage and owner	Root cause	Correction	Reusable lesson
Arm too high/low or ahead/behind; repeated coordinate lowering [LOG, exact sequence missing]	Pre-grasp/approach; geometry and frames	[INFERENCE] Camera/base frame mismatch, wrong hand offset, trusted Z, or model origin	Use one base frame; derive G/P/L equations; synchronize cylinder X/Y; tune one value at a time	A pose error is often a representation error, not an IK error
Camera target and fixed Movelt cylinder could disagree [CODE]	Scene setup; Planning Scene	V1 published [0.60,0.00,0.04] before vision	V2 publishes cylinder at [target_x,target_y,0.04] and stores original pose	Every subsystem must point to the same object before planning
APPROACH completion 0.608 < 0.98 [LOG]	Cartesian planning	Path became unreachable/colliding before goal; exact test geometry not fully recovered	Abort; inspect pose/frame/collision/ACM and replan after geometric correction	Never execute a partial critical approach just because most of it exists
Finger/bottle or bottle/table collision rejection [CODE/LOG]	ACM / Planning Scene	Expected physical contact was treated as forbidden collision	Allow only the required link-object pairs at the required stage	Do not disable all collision checking to solve one expected contact
No bridge subscriber or no attached feedback [CODE messages]	Gazebo attachment	Command published before bridge ready, lost message, or plugin failure	Check subscription count; retry every 0.5 s; require attached/detached feedback	A sent command is not a completed state transition
Gazebo attached but Movelt did not [CODE message]	Dual state	Physical and planning attachment pipelines are independent	Poll attached/world object lists and forbid lift until both agree	Maintain separate truth models and verify both
Movelt detach not confirmed or Gazebo detach fails [CODE]	Placement/detachment	Partial state transition	Abort before release/retreat; inspect both state owners	Add rollback/recovery in future production code

6.2 Trajectory planning, timing, and controller errors

Symptom / exact evidence	Owner	Cause	Final correction	Engineer interpretation
'Joint states are not available yet!' then immediately 'available now' [LOG]	MoveIt/PyMoveIt2 state monitor	Asynchronous startup/update race	Wait/spin until state arrives	Warning is temporary if followed by availability and planning succeeds; fatal if timeout/None persists
[MIX FAILED] Current joint state is unavailable [V2 CODE]	V2 shake start state	self.arm.joint_state remained None after 3 s	Abort instead of forming an invalid request	No current state means no trustworthy trajectory start
MoveIt completed only fraction < 0.999 [CODE]	Cartesian planner	Some waypoint cannot be connected under IK/collision/joint constraints	Reject entire shake; reduce/fix geometry or orientation	Path fraction is a geometric diagnostic, not a number to hide by lowering threshold
Long delay; thousands of interpolated/IK points [LOG/CODE comment]	Cartesian interpolation	0.0025 m max_step over long dense path	Use 0.01 m for open-air shake	Measure planning time and point count before increasing CPU or waiting longer
'path requires a 180 degree turn' [LOG]	TOTG/time parameterization	Abrupt orientation reversal/cusp	Continuous quaternion wrist rock with $\sin(2\phi)$	Smooth pose samples must also produce a smooth joint-space direction
[MIX FAILED] Trajectory timestamps are not strictly increasing [CODE]	Time parameterization	Equal/decreasing time_from_start	Reject before execution	Geometric success is not controller-executable success
[MIX FAILED] Faster trajectory timestamps are not strictly increasing [CODE]	Manual retiming	Compression/rounding/hold insertion broke ordering	Recheck after modification	Validate every representation after transforming it
[MIX FAILED] Trajectory was not time-parameterized [CODE]	Planner pipeline	Last duration ≤ 0	Abort	A path without valid time is not a trajectory
Start point deviates from current robot state >0.01; panda_joint2 then panda_joint7 [LOG]	Controller start tolerance	Explicit/cached start or planned first point lagged actual joints	0.25 s sync; leave start_state empty; endpoint hold for final wrist	Do not relax tolerance before fixing stale state or settling
Shake remained slow at scaling 1.0 [LOG]	Time parameterizer/limits	1.0 is maximum configured scaling; path/reversals remain limiting	0.55 simulation retiming	Scaling and actual Cartesian speed are different quantities
Placement failed after fast shake [LOG]	Post-execution state	panda_joint7 not settled or final state update not consumed	0.45 s endpoint hold + 0.30 s ROS spin	Execution result and settled state are related but not identical

6.3 Build/environment error: correct source but old installed program

[LOG] The executed node still printed **VIGOROUS MIX** after the source had been changed to **LONG FAST SHAKE**. That symptom belongs to the ROS build/environment layer, not motion planning.

```
# 1. Find which copies contain each distinctive log string.
rg -n "VIGOROUS MIX|LONG FAST SHAKE" src build install

# 2. Confirm source syntax before rebuilding.
python3 -m py_compile src/<package>/<package>/bottle_shake_task.py

# 3. Rebuild the owning package and create source-linked installs.
colcon build --symlink-install --packages-select hv_manipulation

# 4. Source THIS workspace in the terminal that will run the node.
source install/setup.bash

# 5. Check which installation ROS resolves.
ros2 pkg prefix hv_manipulation
echo "$AMENT_PREFIX_PATH"

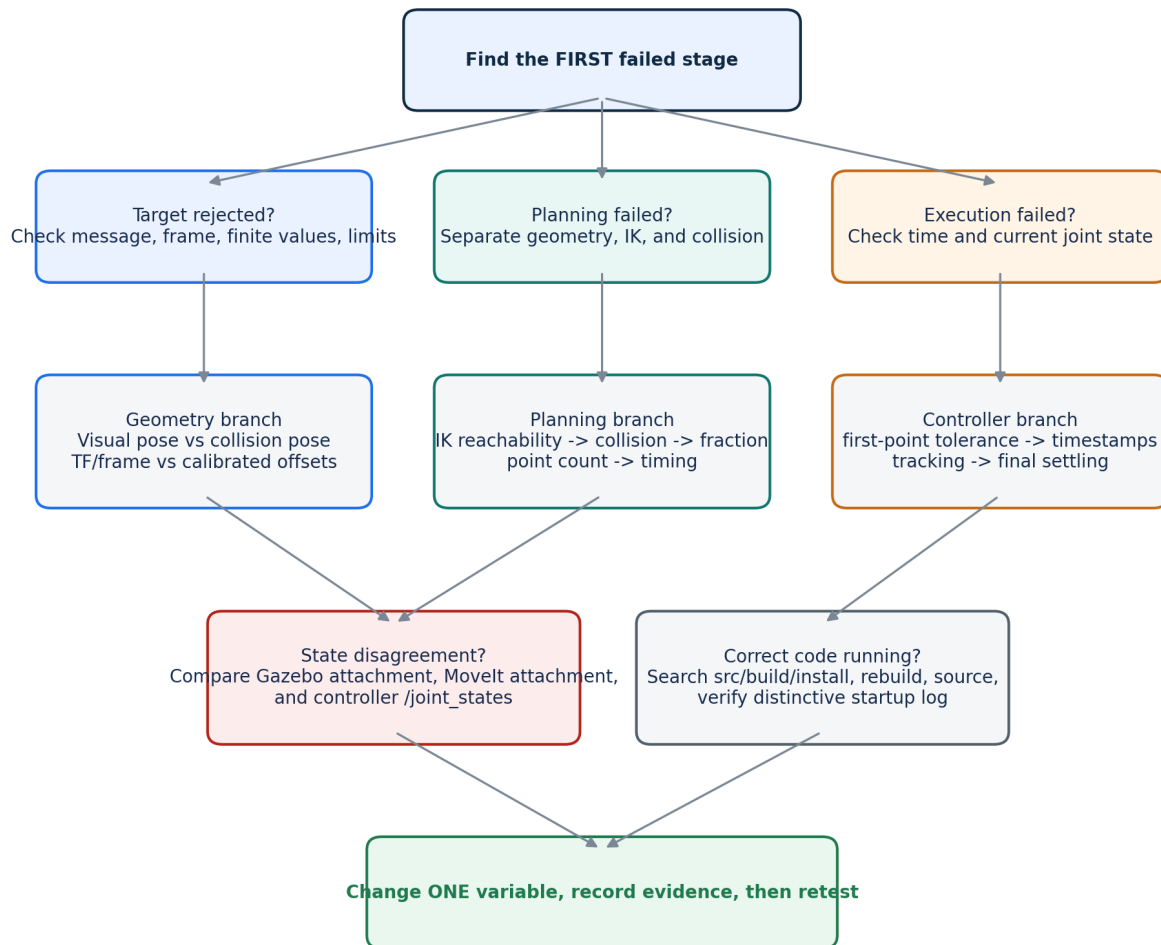
# 6. Run and verify the distinctive expected startup/log message.
ros2 run hv_manipulation bottle_shake_task --ros-args -p use_sim_time:=true
```

Evidence boundary

The stale-log symptom is recovered from project history. The command sequence above is the correct reusable investigation procedure requested for this handbook; the exact historical terminal transcript for every command was not recoverable.

7. Reusable engineering method for building from scratch

Engineer Debugging Decision Tree



7.1 Design around invariants, not around motion commands

Before stage	Invariant that must be true	Evidence to inspect	If false
Planning target	Pose is finite, in the correct frame, and inside intended workspace	Stamped frame/TF, printed target, RViz marker	Reject before IK
Pre-grasp	Static and dynamic collision objects are present at correct poses	Planning Scene query/RViz	Do not move
Approach	Expected finger contact is allowed; table/body collision is still forbidden	ACM pair checks	Fix collision policy, not global avoidance
Lift	Gripper command succeeded; Gazebo and MoveIt both report attached	Action result + dual-state query	Abort/rollback
Shake execution	Full Cartesian fraction, valid timed points, correct current start state	Fraction, point count, duration, first-point delta	Do not execute
Placement	Shake ended at centre/base orientation and controller state is settled	Final waypoint, /joint_states delta	Hold/wait until convergence
Release	Bottle is at support pose; both representations detached	Scene + Gazebo feedback	Do not open/retreat

7.2 The first-failure rule

If APPROACH fails, later gripper, attachment, lift and shake failures are meaningless. If execution rejects the first point, changing bottle coordinates is unlikely to help. Always classify the first failure into one owner: perception/frame, geometry, IK, collision, Cartesian interpolation, timing, controller state, Gazebo physics, or build environment.

Do not debug by symptom location alone

An arm that visually stops before the bottle may be suffering from a collision object at the wrong pose, an ACM rule, a partial Cartesian path, or a hand-frame offset. Ask which subsystem refused progression and read its exact log before changing coordinates.

7.3 Safe one-variable experimental method

Step	Action	Record
1. Freeze baseline	Save source revision and launch commands	Git commit/hash or copy, parameter overrides, sourced workspace
2. Reproduce once	Run without changing anything	First failed stage and exact log
3. Classify owner	Geometry vs planning vs timing vs controller vs physics	Why that owner is responsible
4. Change one variable	Example: grasp Z OR side depth OR max_step, never all three	Old/new value and physical units
5. Measure	Do not rely only on visual impression	Fraction, requested waypoints, trajectory points, planning duration, execution duration
6. Verify start/end	Compare desired and actual state	Per-joint delta; final hand/bottle pose
7. Preserve successful fix	Add validation/log/comment explaining why	The failure class prevented by the code
8. Increase stress gradually	Amplitude first at low speed, then speed, then orientation	First unsafe/unreliable boundary

7.4 Recommended test record

Run	Changed parameter	Target pose/frame	Fraction	Req./traj. points	Plan/exec time	First error	Result
001	baseline	x,y,z in panda_link0					
002	one change only						
003	one change only						

7.5 Geometry-first decision rule

Relaxing a tolerance is appropriate only after proving the target is physically correct and the remaining error is numerical/noise-sized. If the gripper is centimetres too high, a 5 mm tolerance is not the problem. If the collision cylinder is at the wrong X/Y, lowering the Cartesian fraction threshold only permits an unsafe incomplete path.

```
Correct order for a Cartesian failure:
1. confirm frame and target pose
2. confirm collision-object pose/dimensions
3. confirm expected ACM contacts
4. test endpoint IK/reachability
5. inspect fraction and first unreachable waypoint
6. inspect joint-space continuity/jump
7. inspect time parameterization
8. only then reconsider thresholds/resolution
```

7.6 Mechanical interpretation of the fast shake

Concept	Meaning in this project	Engineering implication
7-DOF redundancy	Many joint configurations can realize one hand pose	IK can switch branches; obstacle avoidance and wrist motion redistribute joint demand
Velocity	Rate of joint/hand position change	Higher speed increases tracking difficulty but not by itself the reversal force
Acceleration	Rate of velocity change	Payload force magnitude follows $F = m \cdot a$; time compression multiplies planned acceleration by about 3.306
Jerk	Rate of acceleration change	Abrupt direction/orientation cusps stress time parameterization and mechanics
Moment load	Bottle centre of mass is offset from gripper	Wrist torque roughly $\tau = r \times F$; long fast strokes magnify it
Friction grasp	Two fingers must resist gravity and shake force	Approximate condition $2 \cdot \mu \cdot N \geq m \cdot (g + \text{worst upward acceleration})$
Gazebo attachment	Detachable joint behaves like a rigid constraint	It hides realistic slip, finger compliance and force limits
Movelt attachment	Collision geometry travels with panda_hand	It models occupied space, not physical gripping force

Hardware boundary

For a real Panda: remove manual time compression, use verified URDF/YAML joint limits, start with small amplitudes and low scaling, configure collision/force safety, use force-aware gripper control, validate payload mass/inertia, and provide an emergency stop. A successful Gazebo trajectory is not a hardware approval.

8. What remains fragile in the final V3 code

V3 is substantially stronger than V1, but a good engineer records remaining assumptions instead of calling the program universally robust.

Remaining weakness	Why it matters	Recommended improvement
/color_coordinates is an unframed String	No timestamp/frame; parsing is fragile; wrong optical/base frame is easy	Publish PoseStamped or a custom detection message and transform with tf2
First matching detection is locked	One noisy/outlier frame becomes the full task target	Median/mean several stable detections; check covariance and freshness
Camera Z is validated but ignored	Requires fixed upright bottle/table calibration; cannot handle variable height or lying bottle	Use calibrated transformed Z or derive centre from known table and detected bottle geometry
Workspace limits check only bottle centre X/Y	Derived pre-grasp and +/-0.12/0.15 shake may exceed reach/collision-free region	Validate all derived extrema and optionally pre-check endpoint IK
wait_for_world_bottle_pose confirms presence, not numeric agreement	Stored MoveIt pose is logged but not compared to expected tolerance	Calculate position error and require each axis within a small tolerance
side_grasp_depth and grasp-height offset are fixed calibration constants	Robot/tool/bottle changes invalidate them	Derive from tool transform, finger geometry, bottle radius/height and desired contact band
Simplified cylinder and conservative table overlap	Visual/collision geometry may disagree; ACM can mask excessive overlap	Measure actual model dimensions and inspect collision geometry in RViz
Gazebo rigid attachment masks slip	A physically impossible grasp can still transport the bottle	Use contact/friction/force-based grasp validation for realism
No force, torque, slip or bottle-pose feedback	Successful finger position does not prove secure grasp	Add contact sensors, effort/force checks or visual post-grasp verification
All Cartesian jump thresholds are zero	No explicit nonzero discontinuity limit is enforced by this request	Inspect joint deltas and configure a tested jump threshold where supported
0.55 manual time compression	May violate configured velocity/acceleration/jerk and controller tracking limits	Use supported retiming or audit every joint after retiming; remove on hardware
0.25/0.30 s synchronization is time-based	A slow/loaded system may not settle within fixed time	Wait for fresh JointState sequence/timestamp and numeric convergence thresholds
Endpoint hold commands zero velocity but does not verify actual convergence	Controller success can precede exact settled state	Compare measured joints against final positions before placement
No automatic rollback after partial attach/detach	Abort may leave Gazebo and MoveIt inconsistent	Implement recovery states: stop, query both, detach/recreate object, return safe pose
Generic Cartesian threshold remains 0.98	A 98.x% critical approach/lift/place can still be incomplete	Use stage-specific thresholds and verify final pose error
Large monolithic class/file	Harder testing and reuse; transport, scene and shake logic are coupled	Split PerceptionAdapter, SceneManager, AttachmentManager, MotionExecutor and TaskStateMachine
Unused/legacy items	drop_joints, gripper_closed and remove_bottle_collision obscure intent; execute_day1_pick_place name is outdated	Remove or document; rename task method for final purpose
No recovery/replan policy	Any transient failure ends the task and may require manual reset	Add bounded retries only for classified transient failures plus a safe-stop path

8.1 Which issues are acceptable for this simulation demonstration?

For the completed professor demonstration, fixed upright bottle geometry, rigid Gazebo attachment, a monolithic node, and time-based settling are understandable. They become unacceptable when the goal changes to arbitrary bottle poses, realistic slip, moving targets, physical hardware, or autonomous recovery.

Do not confuse scope with quality

An engineering solution can be correct for a clearly bounded simulation task while still documenting assumptions. Robustness means the code detects when those assumptions are violated; it does not mean solving every possible bottle and environment.

9. Blueprint for the next manipulation program

Phase	Build	Exit criterion
1. Define frames and geometry	URDF/SRDF/tool frame, table/object collision primitives, TF contract	One known pose is correct in RViz and Gazebo
2. Prove interfaces	Joint states, arm action, gripper action, Planning Scene services, attachment feedback	Each interface independently confirms success/failure
3. Implement state ownership	Perception target, MoveIt object, Gazebo object, controller state	Queries show all owners agree before motion
4. Implement safe pick	HOME, pre-grasp, ACM, Cartesian approach, close, dual attach, lift	Repeatable pick with fail-fast logs
5. Implement place	Above pose, slow contact, dual detach, release, retreat	Bottle stable and all states restored
6. Add simple motion	Small continuous Cartesian path at low scaling	fraction, timing and execution measured
7. Increase complexity	Amplitude, cycles, speed, then orientation - one at a time	Known safe/reliable envelope recorded
8. Add recovery	Transient retry, state resync, rollback and safe-stop	No failure leaves unknown attachment/state
9. Prepare hardware	Limit audit, payload/inertia, force safety, E-stop, low-speed validation	Separate hardware sign-off

9.1 Design checklist

- Every pose has a named frame and timestamp.
- Object visual, collision and physical dimensions are measured and compared.
- The tool-frame offset to the intended contact point is derived or calibrated explicitly.
- All derived poses and path extrema are validated, not only the object centre.
- Expected collision pairs are allowed only for the stage that needs them.
- Physical and planning attachment states are queried independently.
- Every plan checks nonempty output, fraction/final error and timing validity.
- The trajectory start matches fresh measured state within controller tolerance.
- Requested waypoint count, generated trajectory points, plan time and execution time are logged.
- Speed experiments increase one variable at a time from a conservative baseline.
- Any manual trajectory edit is followed by timestamp and per-joint limit validation.
- Final measured state is settled before planning the next trajectory.
- Every abort has a defined safe state or recovery procedure.
- The running ROS package/version is identifiable from a distinctive log and package prefix.
- Simulation-only assumptions are explicitly removed or revalidated before hardware.

9.2 Ten questions that test real understanding

- 1. Why can Gazebo show the bottle moving correctly while MoveIt still refuses a plan?
- 2. Why is side_grasp_depth not equal to the bottle radius?
- 3. Why should PRE-GRASP use free-space planning but APPROACH use Cartesian planning?
- 4. What does a Cartesian fraction of 0.608 prove, and what does it not prove?

- 5. Why did one continuous V2 trajectory look smoother even though its scaling values matched V1?
- 6. Why can `self.arm.joint_state` be unsafe as an explicitly copied start state immediately after motion?
- 7. How does `sin(2phi)` make the wrist continue moving at a translational reversal?
- 8. Why does $t' = 0.55t$ require $v' = v/0.55$ and $a' = a/(0.55^2)$?
- 9. Why is an ACM permission unrelated to Gazebo friction or gripping force?
- 10. Which final-code assumptions must change before picking a lying bottle or running on hardware?

10. Glossary and source notes

Term	Meaning in this project
DOF	Degree of freedom; Panda has seven independently controlled arm joints
IK	Inverse kinematics; convert hand pose to joint angles
URDF	Robot link/joint geometry, origins and physical/configured limits
SRDF	Semantic planning groups, end effectors, named states and collision semantics
TF / tf2	ROS transform tree used to express one frame in another
Planning Scene	Movelt's robot/world/attached-object/collision state
ACM	Allowed Collision Matrix; selected contacts permitted during planning
RRTConnect	Sampling-based free-space planner used for pose/joint travel
Cartesian path	End-effector path sampled through workspace and solved through IK
Waypoint	One desired intermediate Pose; not necessarily one controller trajectory point
Trajectory	Joint positions with valid <code>time_from_start</code> and optional velocity/acceleration
TOTG	Time-Optimal Trajectory Generation; assigns feasible trajectory timing
Start state	Robot joint state from which planning/trajectory execution begins
Scaling factor	Fraction of configured joint velocity or acceleration limits
Quaternion	Four-component unit representation of 3-D rotation; code uses XYZW order
Redundancy	Multiple seven-joint solutions for one six-dimensional hand pose

10.1 Evidence and limitations

This handbook was derived from the three verified source files, the recovered project conversations/logs, and direct code comparison. Exact old coordinate-by-coordinate trial logs were not fully recoverable. They are therefore summarized only as confirmed repeated calibration symptoms, while the proposed causes are marked as engineering inference.

Recovered exact evidence includes: V1's 17 shake calls; a Cartesian completion of 0.608 below 0.98; V2's successful fraction=1.000, 147 trajectory points, 14.59 s and 0.27 Hz; temporary joint-state warnings followed by availability; the '180 degree turn' report; non-increasing timestamp checks; start-state deviation above 0.01 involving `panda_joint2/panda_joint7`; planning delay from dense interpolation; slow motion at scaling 1.0; placement failure after a fast shake; and the VIGOROUS MIX versus LONG FAST SHAKE stale-build symptom.

Final engineering takeaway

The most reusable lesson is not a coordinate or a Movelt API call. It is the discipline of maintaining consistent state across perception, geometry, planning, controller execution and physics - then changing the smallest responsible subsystem when the first invariant fails.